



Microcontrollers

Rex Joseph

Department of Electrical and
Electronics Engineering

Recap

Generic Microprocessor/microcontroller

FMSD!

Controller plus data-path

Components that make up the uC

CPU (ALU plus registers)

Memory

Peripherals

Clock

Address and data bus

Architectural features, instruction set

Harvard Von Neumann, RISC CISC

Development board (Hardware)

Development environment (Software) IDE



Topics

Processor Package
Internal organization

Ports and pin map

Key features

Architecture

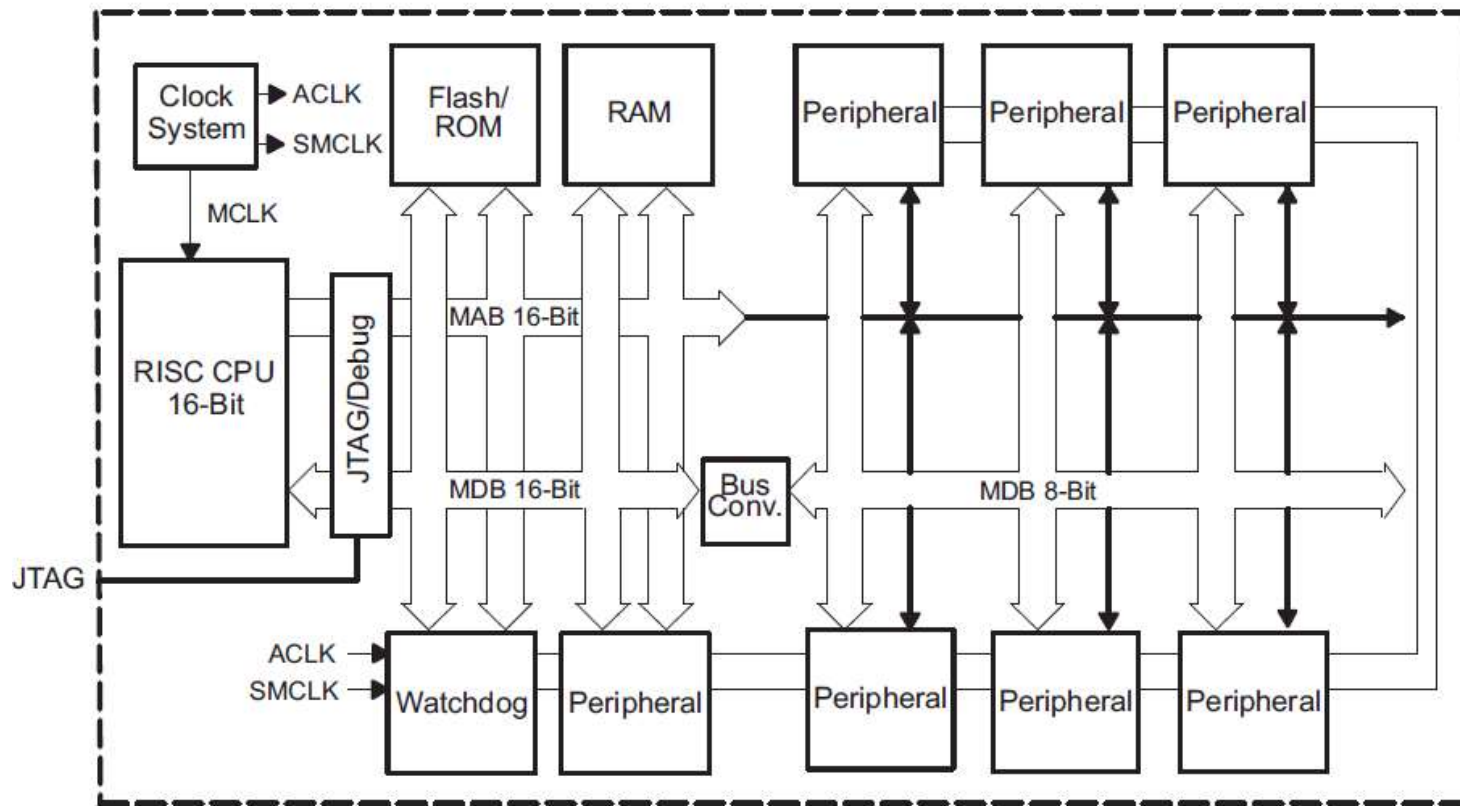
- CPU

- Memory

- CPU Registers

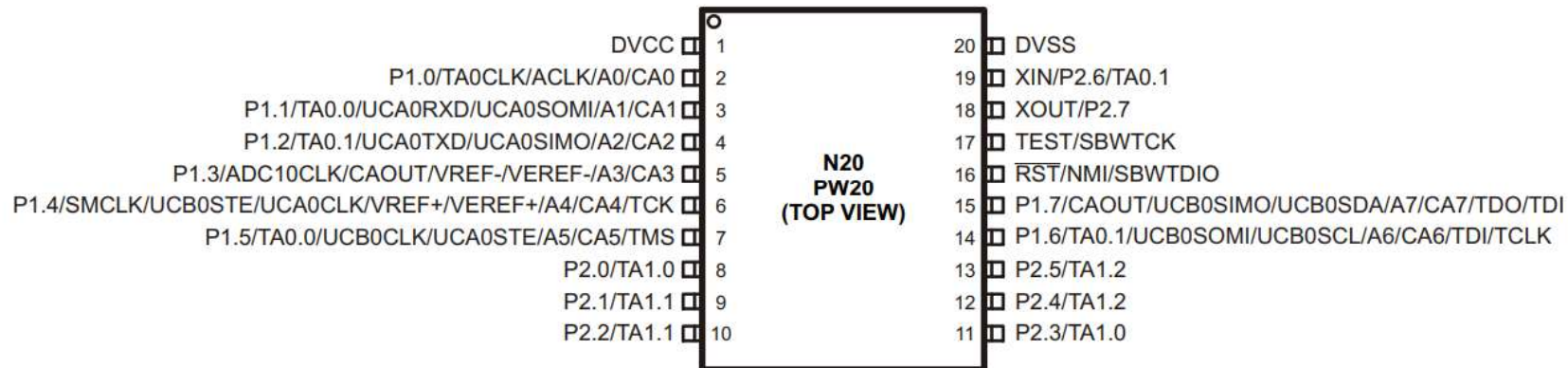


MSP430 Architecture



MSP430 Architecture

Device Pinout, MSP430G2x13 and MSP430G2x53, 20-Pin Devices, TSSOP and PDIP

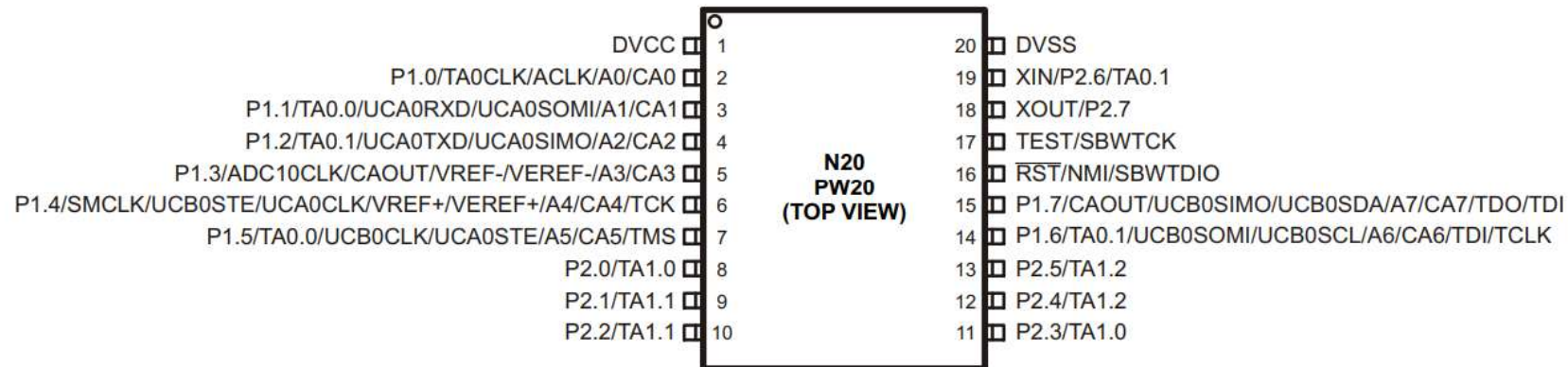


NOTE: ADC10 is available on MSP430G2x53 devices only.

NOTE: The pulldown resistors of port P3 should be enabled by setting P3REN.x = 1.

MSP430 Architecture

Device Pinout, MSP430G2x13 and MSP430G2x53, 20-Pin Devices, TSSOP and PDIP



NOTE: ADC10 is available on MSP430G2x53 devices only.

NOTE: The pulldown resistors of port P3 should be enabled by setting P3REN.x = 1.

Thin Shrink Small Outline Package / Plastic Dual Inline Package

MSP430 Architecture

FEATURES

Low Supply-Voltage Range: 1.8 V to 3.6 V

Ultra-Low Power Consumption

- Active Mode: 230 μ A at 1 MHz, 2.2 V
- Standby Mode: 0.5 μ A
- Off Mode (RAM Retention): 0.1 μ A

Five Power-Saving Modes

Ultra-Fast Wake-Up From Standby Mode in C[™] Less Than 1 μ s

16-Bit RISC Architecture, 62.5-ns Instruction Cycle Time

Basic Clock Module Configurations

- Internal Frequencies up to 16 MHz With Converter With Four Calibrated Frequencies
- Internal Very-Low-Power Low-Frequency (LF) Oscillator
- 32-kHz Crystal
- External Digital Clock Source

MSP430 Architecture

FEATURES

Two 16-Bit Timer_A With Three Capture/Compare Registers

Up to 24 Capacitive-Touch Enabled I/O Pins

Universal Serial Communication Interface (USCI)

- Enhanced UART Supporting Auto Baudrate Detection (LIN)
- IrDA Encoder and Decoder
- Synchronous SPI
- I2C

On-Chip Comparator for Analog Signal Compare Function or Slope Analog-to-Digital (A/D) Conversion

10-Bit 200-ksps Analog-to-Digital (A/D) converter with Internal Reference, Sample-and-Hold, and Autoscan

Brownout Detector

Serial Onboard Programming, No External Programming Voltage Needed

MSP430 Architecture

FEATURES

Programmable Code Protection by Security Fuse

On-Chip Emulation Logic With Spy-Bi-Wire Interface

Package Options – TSSOP: 20 Pin, 28 Pin – PDIP: 20 Pin – QFN: 32 Pin

For Complete Module Descriptions, See the MSP430x2xx Family User's Guide (SLAU144)

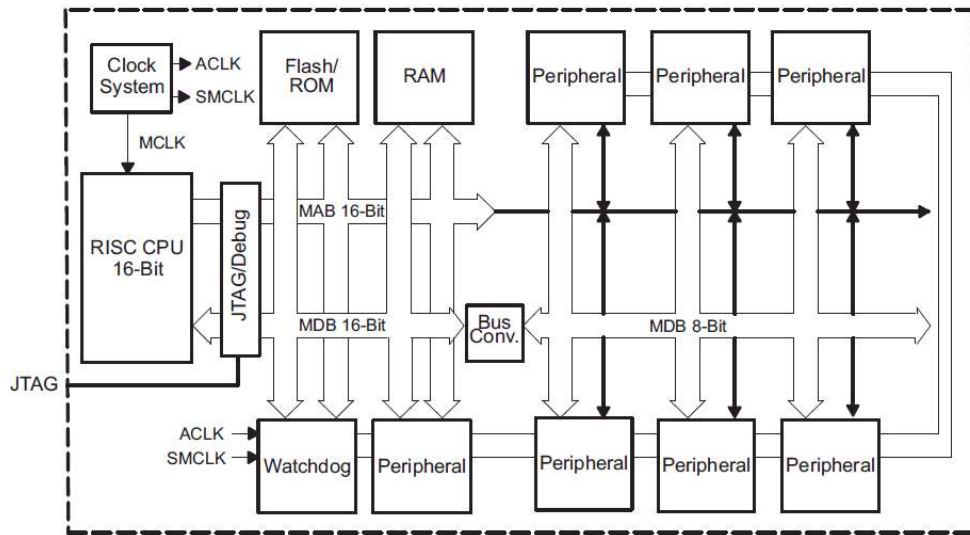
The MSP430 incorporates a **16-bit RISC CPU**, peripherals, and a flexible clock system that interconnect using a **von-Neumann** common memory address bus (MAB) and memory data bus (MDB)

There is no separate memory for code and data. Instructions and data are interleaved in the memory space

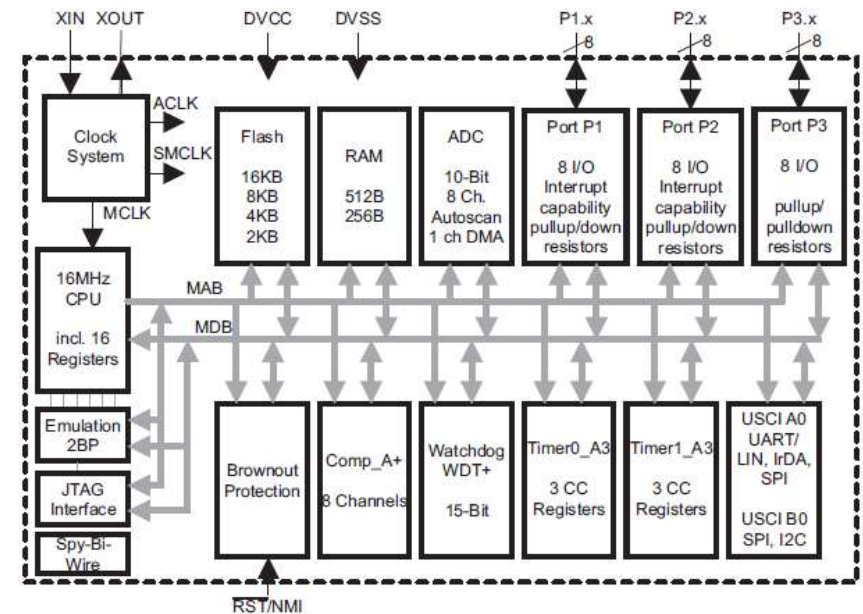
MSP430 Architecture

- 16-bit RISC CPU enables new applications at a fraction of the code size.
 - Large register file eliminates working file bottleneck
 - Compact core design reduces power consumption and cost
 - Optimized for modern high-level programming
 - Only **27 core instructions** and seven **addressing modes**
 - **Extensive vectored-interrupt capability**
- In-system programmable Flash permits flexible code changes, field upgrades and data logging

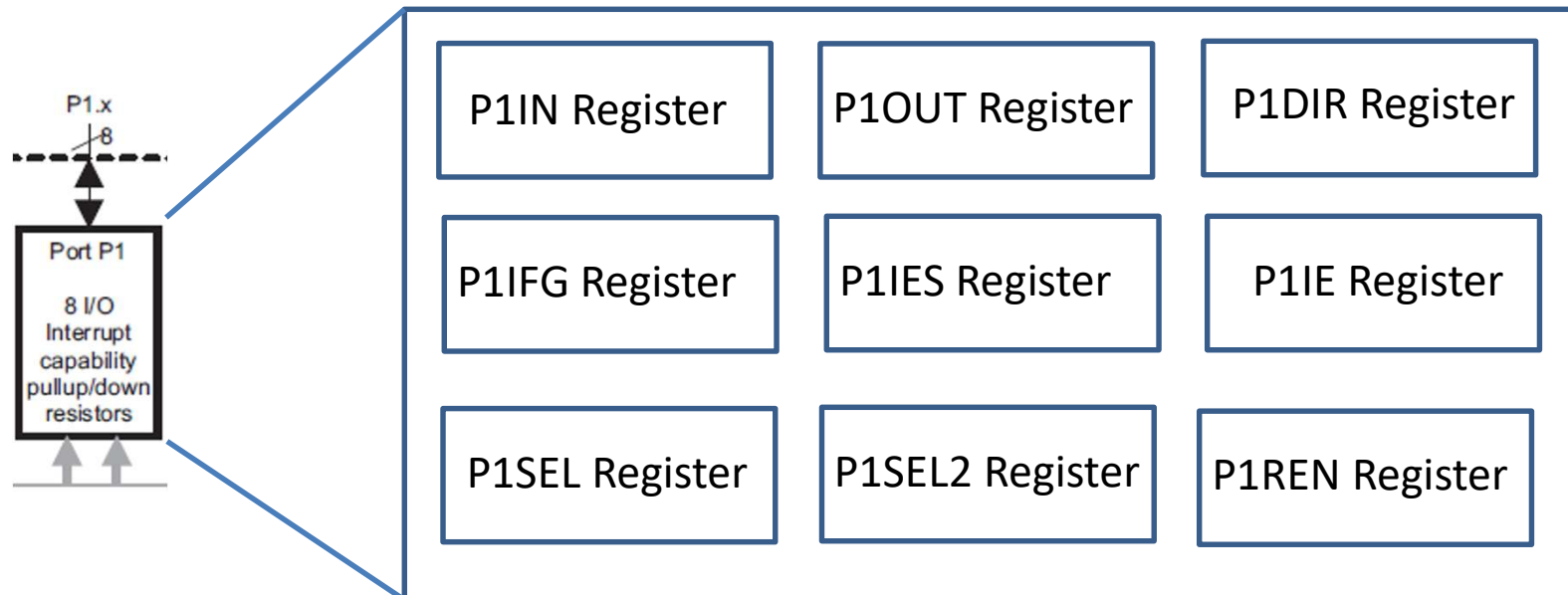
MSP430 Architecture



Functional Block Diagram, MSP430G2x53



Port 1 Registers



Need to initialize some/all of these registers for proper operation

Default values would be sufficient for some applications

Port 1 Registers

8.3.3 PxDIR Register

Port x Direction Register

Figure 8-5. PxDIR Register

7	6	5	4	3	2	1	0
PxDIR							
rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0

Table 8-5. P1DIR Register Description

Bit	Field	Type	Reset	Description
7-0	PxDIR	R/W	0h	Port x direction. Each bit corresponds to one channel on Port x. 0b = Port configured as input 1b = Port configured as output

```

59  P1DIR |= 0x01;
60
61  while(1)
62  {
63      P1OUT ^= 0x01;
64
65      for (i=10000; i>0; i--);
66  }
67

```

1.7	1.6	1.5	1.4	1.3	1.2	1.1	1.0
0	0	0	0	0	0	0	1

Why OR rather than write the value? (0x01)

Portx Registers

Table 8-2. Digital I/O Registers

Port	Address	Acronym	Register Name	Type	Reset	Section
P1	020h	P1IN	Input	Read only	Unchanged	Section 8.3.1
	021h	P1OUT	Output	Read/write	Unchanged	Section 8.3.2
	022h	P1DIR	Direction	Read/write	00h with PUC	Section 8.3.3
	023h	P1IFG	Interrupt Flag	Read/write	00h with PUC	Section 8.3.4
	024h	P1IES	Interrupt Edge Select	Read/write	Unchanged	Section 8.3.5
	025h	P1IE	Interrupt Enable	Read/write	00h with PUC	Section 8.3.6
	026h	P1SEL	Port Select	Read/write	00h with PUC	Section 8.3.7
	041h	P1SEL2	Port Select 2	Read/write	00h with PUC	Section 8.3.8
	027h	P1REN	Resistor Enable	Read/write	00h with PUC	Section 8.3.9
P2	028h	P2IN	Input	Read only	Unchanged	Section 8.3.1
	029h	P2OUT	Output	Read/write	Unchanged	Section 8.3.2
	02Ah	P2DIR	Direction	Read/write	00h with PUC	Section 8.3.3
	02Bh	P2IFG	Interrupt Flag	Read/write	00h with PUC	Section 8.3.4
	02Ch	P2IES	Interrupt Edge Select	Read/write	Unchanged	Section 8.3.5
	02Dh	P2IE	Interrupt Enable	Read/write	00h with PUC	Section 8.3.6
	02Eh	P2SEL	Port Select	Read/write	C0h with PUC ⁽¹⁾	Section 8.3.7
	042h	P2SEL2	Port Select 2	Read/write	00h with PUC	Section 8.3.8
	02Fh	P2REN	Resistor Enable	Read/write	00h with PUC	Section 8.3.9

MSP430 Architecture

Details of registers later – Called control registers or **Special Function Registers (SFRs)**

These registers are mapped to the lower address space

Need to configure them correctly for proper working

And don't forget to disable the watchdog timer
It has a password and a control word

MSP430 Architecture

- **Flexible Clock System**

The clock system is designed specifically for battery-powered applications. A low-frequency auxiliary clock (ACLK) is driven directly from a common 32-kHz watch crystal. Master CLK, Sub MCLK

The ACLK can be used for a background real-time clock self wake-up function.

An integrated high-speed digitally controlled oscillator (DCO) can source the master clock (MCLK) used by the CPU and high-speed peripherals.

By design, the DCO is active and stable in less than 2 μ s at 1 MHz. MSP430-based solutions effectively use the high-performance 16-bit RISC CPU in very short bursts.

- Low-frequency auxiliary clock => Ultralow-power stand-by mode
- High-speed master clock => High performance signal processing

MSP430 Architecture

- **Embedded Emulation**

Dedicated embedded emulation logic resides on the device itself and is accessed via JTAG using no additional system resources.

The benefits of embedded emulation include:

- Unobtrusive development and debug with full-speed execution, breakpoints, and single-steps in an application are supported.
- Development is in-system subject to the same characteristics as the final application.
- Mixed-signal integrity is preserved and not subject to cabling interference.

MSP430 Architecture

- **Address Space**

The MSP430 von-Neumann architecture has one address space shared with special function registers (SFRs), peripherals, RAM, and Flash/ROM memory

Device-specific data sheets for specific memory maps.

Code access are always performed on **even addresses**.

Data can be accessed as **bytes** or **words**.

The addressable memory space is currently 128 KB. (address lines?)

Depends on the specific processor

MSP430 Architecture

Memory Map

1FFFFh	Flash/ROM	Word/Byte
10000h		
0FFFFh	Interrupt Vector Table	Word/Byte
0FFE0h		
0FFDFh	Flash/ROM	Word/Byte
0200h	RAM	Word/Byte
01FFh		
	16-Bit Peripheral Modules	Word
0100h		
0FFh	8-Bit Peripheral Modules	Byte
010h		
0Fh	Special Function Registers	Byte
0h		

MSP430 Architecture

Flash/ROM

The start address of Flash/ROM depends on the amount of Flash/ROM present and varies by device.

The end address for Flash/ROM is 0x0FFFF for devices with less than 64KB of Flash/ROM.

Flash can be used for both code and data.

Word or byte tables can be stored and used in Flash/ROM without the need to copy the tables to RAM before using them.

The **interrupt vector table** is mapped into the upper 16 words of Flash/ROM address space, with the **highest priority interrupt vector** at the **highest Flash/ROM word address (0x0FFFE)**.

RAM starts at 0200h. The end address of RAM depends on the amount of RAM present and varies by device. RAM can be used for both code and data.

MSP430 Architecture

Peripheral Modules

Peripheral modules are mapped into the **address space**.

The address space from 0100 to 01FFh is reserved for **16-bit peripheral modules**.

These modules should be accessed with **word instructions**.

If byte instructions are used, only even addresses are permissible, and the high byte of the result is always 0.

The address space from 010h to 0FFh is reserved for **8-bit peripheral modules**.

These modules should be accessed with **byte instructions**.

Read access of byte modules using word instructions results in unpredictable data in the high byte.

If word data is written to a byte module only the low byte is written into the peripheral register, ignoring the high byte.

MSP430 Architecture

Special Function Registers (SFRs) (aka control registers)

Some peripheral functions are configured in the SFRs.

The SFRs are located in the lower 16 bytes of the address space, and are organized by byte.

SFRs must be accessed using **byte instructions only**.

Device-specific data sheets for applicable SFR bits

MSP430 Architecture

Memory Organization

Bytes are located at even or odd addresses.

Words are **only located at even addresses**

When using word instructions, only even addresses may be used.

The **low byte** of a word is always an even address.

The **high byte** is at the **next odd address**.

If a data word is located at address xxxAh, then the low byte of that data word is located at address ???h, and the high byte of that word is located at address ???h.

MSP430 Architecture

Memory Organization

Bytes are located at **even or odd** addresses.

Words are **only located at even addresses**

When using word instructions, only even addresses may be used.

The **low byte** of a word is always an even address.

The **high byte** is at the **next odd address**.

If a data word is located at address xxx4h, then the low byte of that data word is located at address xxx4h, and the high byte of that word is located at address Xxx5h.

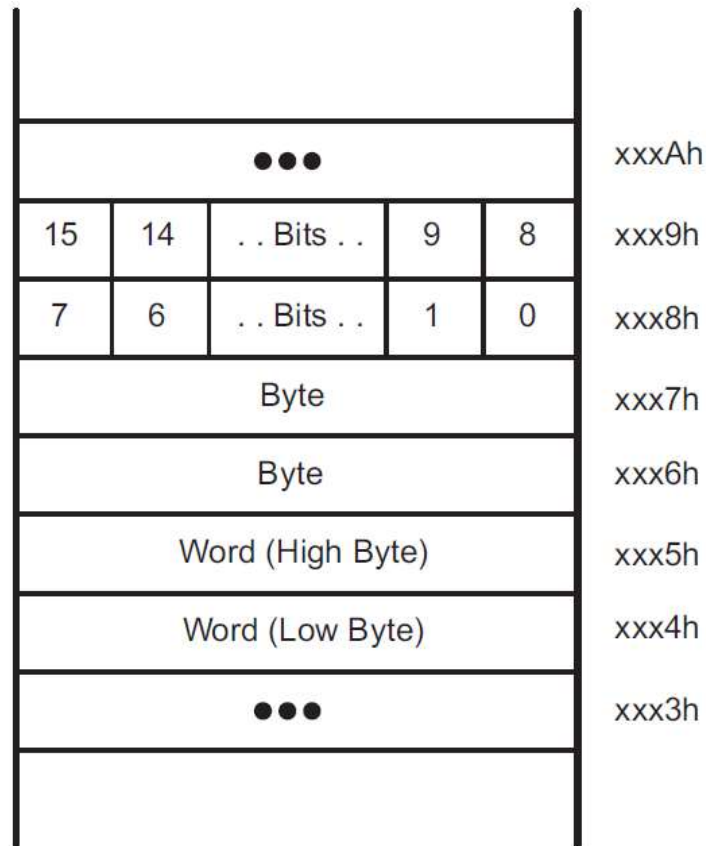
Little-endian ordering: The low-order byte is stored at the lower address and the high-order byte at the higher address. This is used by the MSP430 and is the more common format.

Big-endian ordering: The high-order byte is stored at the lower address. This is used by the Freescale HCS08, for instance.

MSP430 Architecture

The two bytes at xxx8h and xxx9h can be considered as a valid word with address xxx8h, which may be fetched in a single cycle of the bus.

On the other hand, it is not possible to treat the two bytes at xxx9h and xxxAh as a single word because their address would be xxx9h, which is odd and therefore invalid



MSP430 Architecture

CPU

The CPU incorporates features specifically designed for modern programming techniques such as calculated branching, table processing, and the use of high-level languages such as C.

The CPU can address the complete address range without paging.

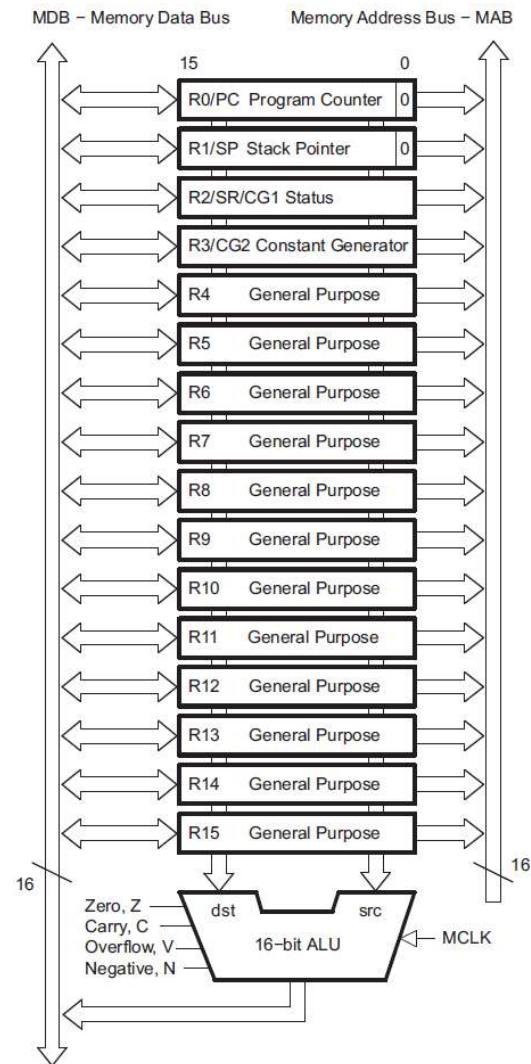
The CPU features include:

- RISC architecture with 27 instructions and 7 addressing modes.
- **Orthogonal** architecture with every instruction usable with every addressing mode.
- Full register access including program counter, status registers, and stack pointer.
- Single-cycle register operations.

MSP430 Architecture

- Large 16-bit register file reduces fetches to memory.
- 16-bit address bus allows direct access and branching throughout entire memory range.
- 16-bit data bus allows direct manipulation of word-wide arguments.
- **Constant generator** provides six most used immediate values and reduces code size.
- Direct memory-to-memory transfers without intermediate register holding.
- Word and byte addressing and instruction formats.

CPU Block Diagram



MSP430 Architecture

CPU Registers

The CPU incorporates sixteen 16-bit registers.

R0, R1, R2, and R3 have **dedicated functions**.

R4 to R15 are working registers for general use

Program Counter (PC)

The 16-bit program counter (PC/R0) points to the next instruction to be executed.

Each instruction uses an even number of bytes (two, four, or six), and the PC is incremented accordingly.

Instruction accesses in the 64-KB address space are performed on word boundaries, **and the PC is aligned to even addresses.**

Program Counter															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Program Counter Bits 15 to 1															0

MSP430 Architecture

The PC can be addressed with **all instructions and addressing modes**.

A few examples:

MOV #LABEL,PC ; Branch to address LABEL

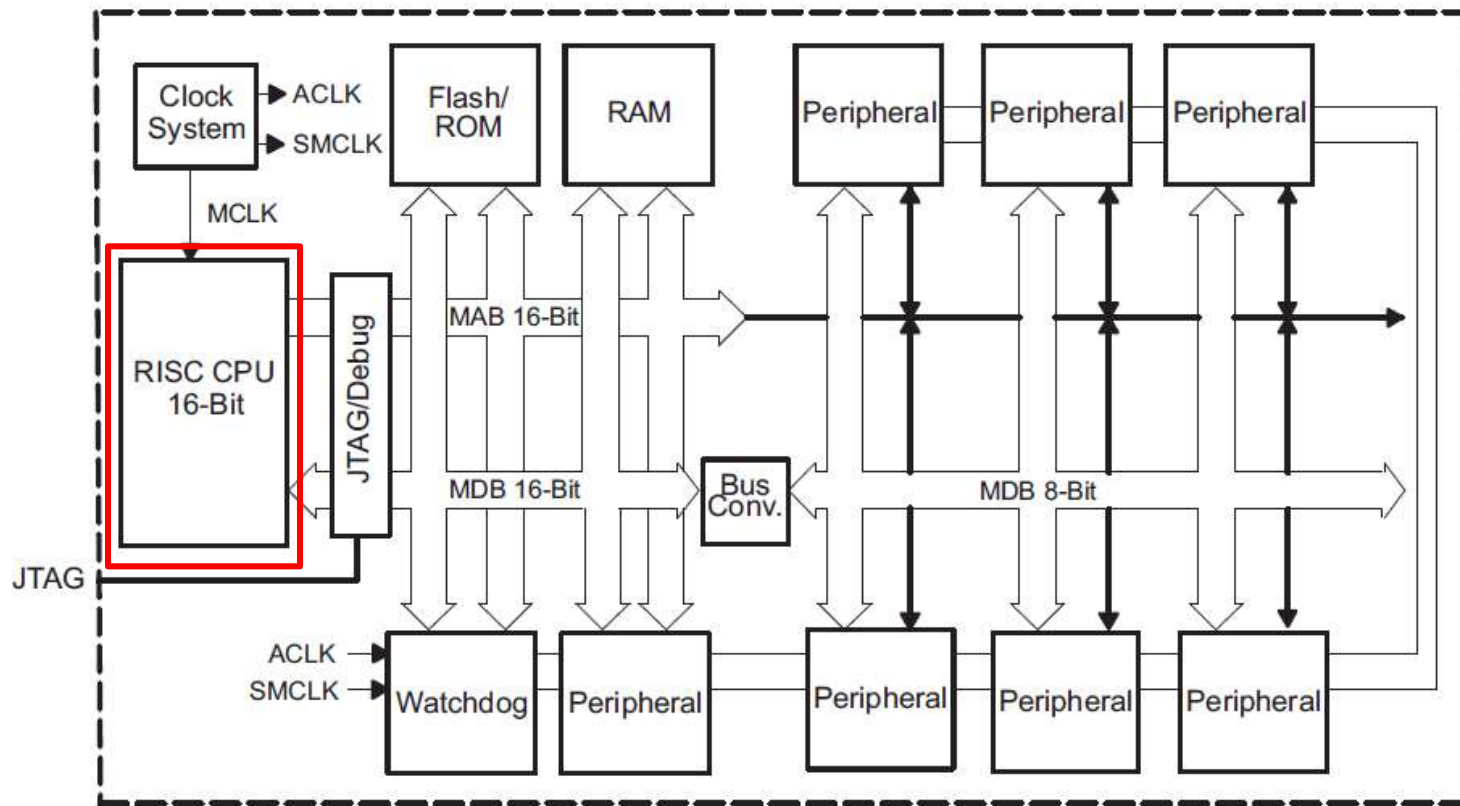
MOV LABEL,PC ; Branch to address contained in LABEL

MOV @R14,PC ; Branch indirect to address in R14

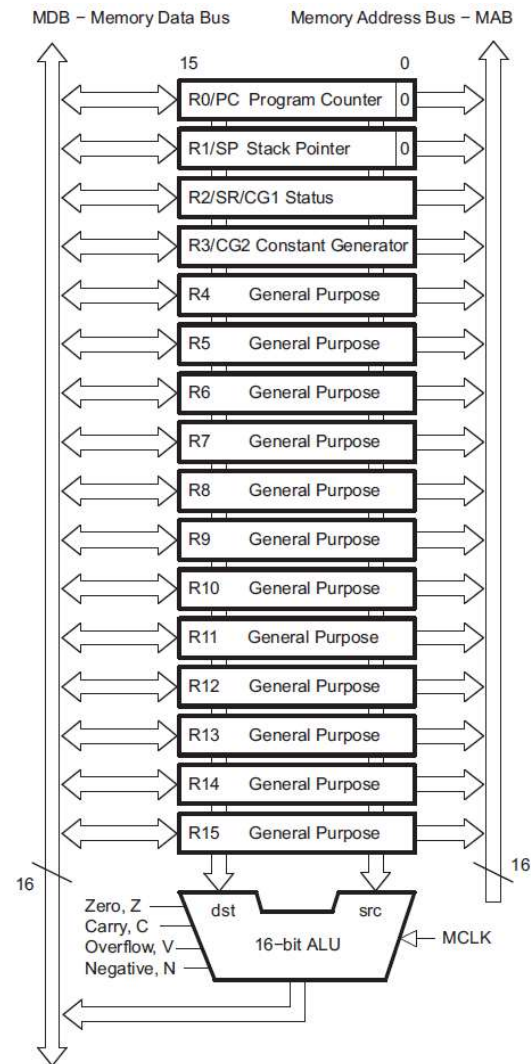
Instructions generally have a source and a destination. There are exceptions

Addressing modes

Recap: MSP430 Architecture



CPU Block Diagram



MSP430 Architecture

Stack Pointer (SP)

The stack pointer (SP/R1) is used by the CPU to store the **return addresses of subroutine calls and interrupts**.

It uses a pre-decrement, post-increment scheme.

In addition, the SP can be used by software with all instructions and addressing modes.

The SP is **initialized into RAM by the user**, and is aligned to even addresses.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Stack Pointer Bits 15 to 1															0

MSP430 Architecture

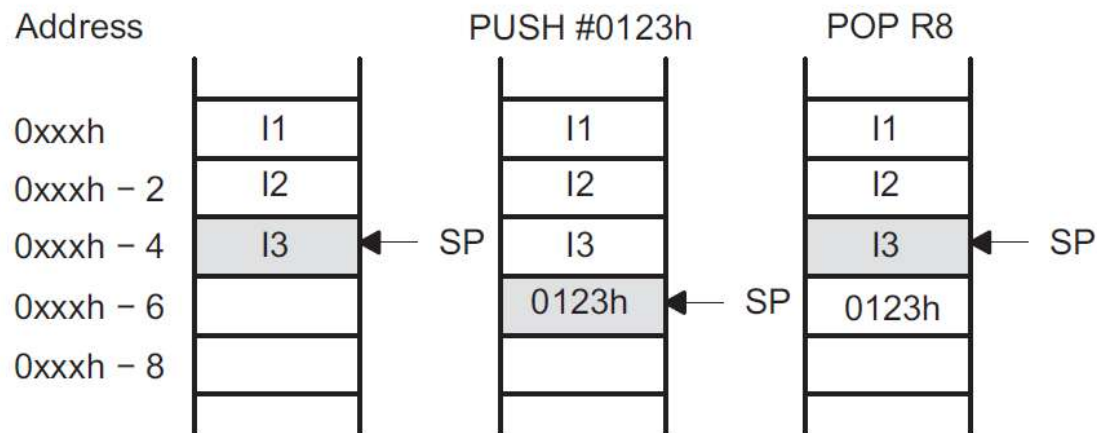
Stack Pointer (SP)

MOV 2(SP),R6 ; Item I2 -> R6

MOV R7,0(SP) ; Overwrite TOS with R7

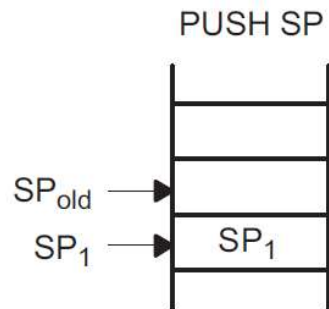
PUSH #0123h ; Put 0123h onto TOS

POP R8 ; R8 = 0123h

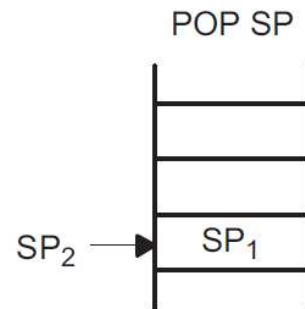


MSP430 Architecture

The special cases of using the SP as an argument to the PUSH and POP instructions



The stack pointer is changed after a PUSH SP instruction.



The stack pointer is not changed after a POP SP instruction. The POP SP instruction places SP1 into the stack pointer SP (SP2=SP1)

MSP430 Architecture

Status Register (SR)

The status register (SR/R2), used as a source or destination register, can be used in the **register mode only** addressed with **word instructions**.

The remaining combinations of addressing modes are used to support the **constant generator**

Status Register Bits

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved							V	SCG1	SCG0	OSC OFF	CPU OFF	GIE	N	Z	C
rw-0							rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0

MSP430 Architecture

Bit Description

V Overflow bit. This bit is set when the result of an arithmetic operation overflows the signed-variable range.

ADD(.B), ADDC(.B)

Set when:

Positive + Positive = Negative

Negative + Negative = Positive

Otherwise reset

SUB(.B), SUBC(.B), CMP(.B)

Set when:

Positive – Negative = Negative

Negative – Positive = Positive

Otherwise reset

SCG1 System clock generator 1. When set, turns off the SMCLK.

SCG0 System clock generator 0. When set, turns off the DCO dc generator, if DCOCLK is not used for MCLK or SMCLK.

MSP430 Architecture

Bit Description

OSCOFF Oscillator Off. When set, turns off the LFXT1 crystal oscillator, when LFXT1CLK is not in use for MCLK or SMCLK.

CPUOFF CPU off. When set, turns off the CPU.

GIE General interrupt enable. When set, enables maskable interrupts. When reset, all maskable interrupts are disabled. (Also called **Global Interrupt Enable**)

N Negative bit. Set when the result of a byte or word operation is negative and cleared when the result is not negative.

Word operation: N is set to the value of **bit 15** of the result.

Byte operation: N is set to the value of **bit 7** of the result.

Z Zero bit. Set when the result of a byte or word operation is 0 and cleared when the result is not 0.

C Carry bit. Set when the result of a byte or word operation produced a carry and cleared when no carry occurred.



THANK YOU

Rex Joseph

Department of Electrical and Electronics Engineering

rexjoseph@pes.edu

+91 80 6666 3333 Extn 709